PETER WOLF
< Web Developer & Advisor />

UseEffect advanced

details about it and more ….

source:

https://www.youtube.com/watch?v=QQYeipc_cik

TODO: https://wanago.io/2022/04/11/abort-controller-race-conditions-react/

First example

we have a button and a message. If we click on the button, the state will update… BUT the <title> update has an delay…

```
const UserSwitching = () => {
    //throttle internet to slow
    const [user, setUser] = useState()
    const [userId, setUserId] = useState(1)
    useEffect(() => {
        const controller = new AbortController()
        const signal = controller.signal
        fetch(`https://jsonplaceholder.typicode.com/users/${userId}`, { signal })
            .then(res => res.json())
            .then(data => setUser(data))

        return () => controller.abort()

    }, [userId])
    function handleChange(e:any) {
        setUserId(e.target.id)
    }
    return (
        <div>
            <div id="1" onClick={handleChange}>User 1</div>
            <div id="2" onClick={handleChange}>User 2</div>
            <div id="3" onClick={handleChange}>User 3</div>
            <p>{JSON.stringify(user)}</p>
        </div>
    )
}
```

PETER WOLF
< Web Developer & Advisor />

Why is the delay:

you have to know, that we have three parties communicating with each other

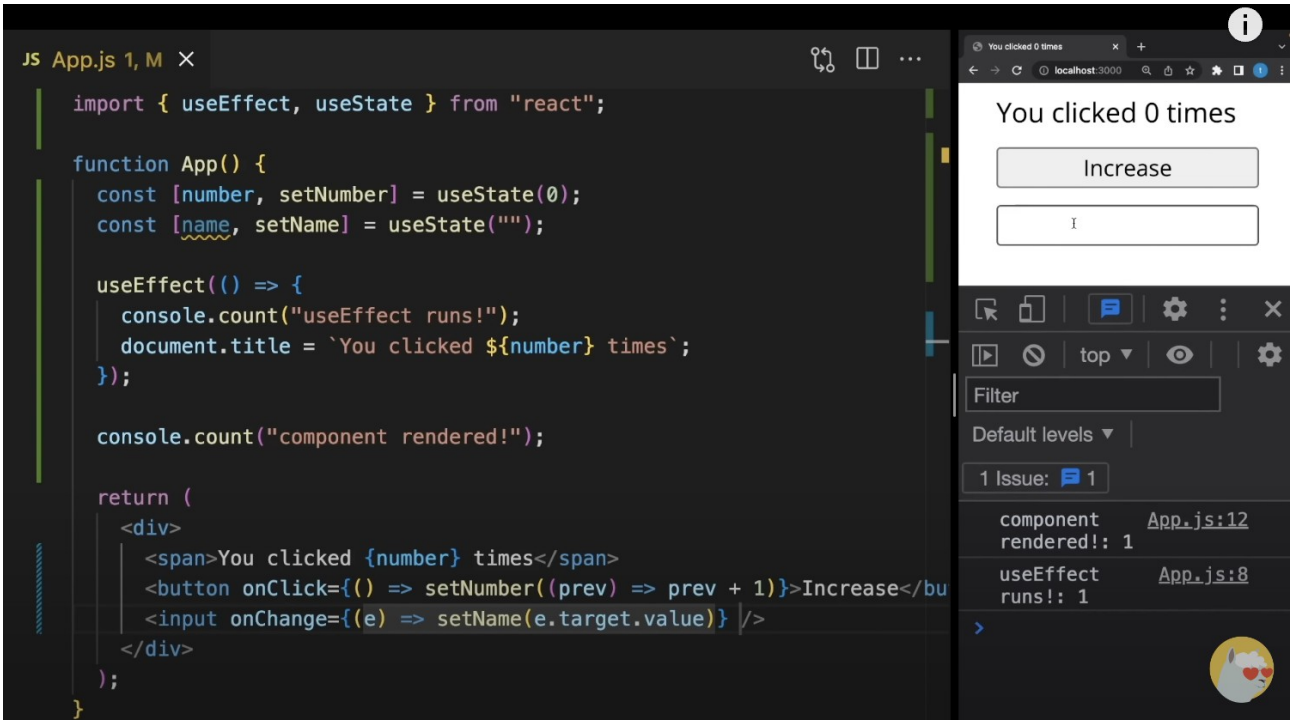
- Component
- React itself
- browser

component says to react, please update
after that, react tells the browser to update the <title>

But each of the updates above triggers a render.

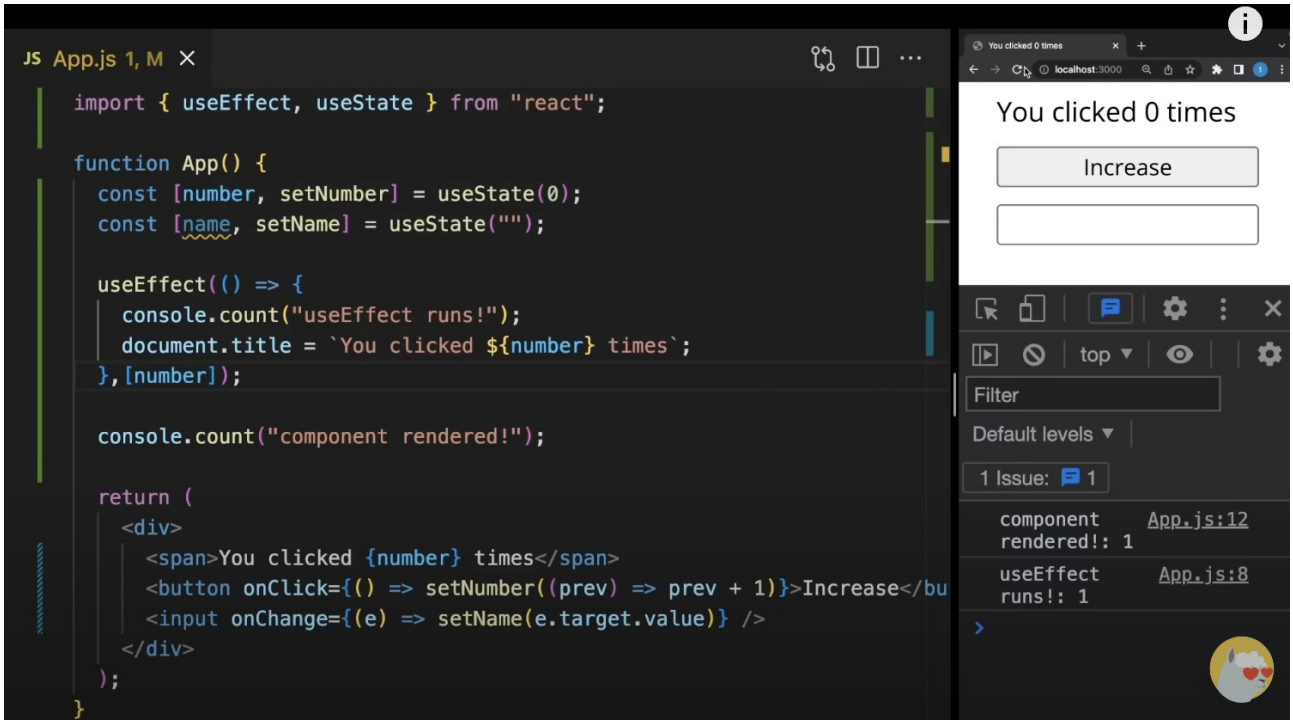
Lets extend the example.

We will now have an input field to update a name state



every time I change the name input, the useEffect will run. But this shouldn´t be happen, because the useEffect depends on the number.

This is the time when you have to use [dependencies] in useEffect.

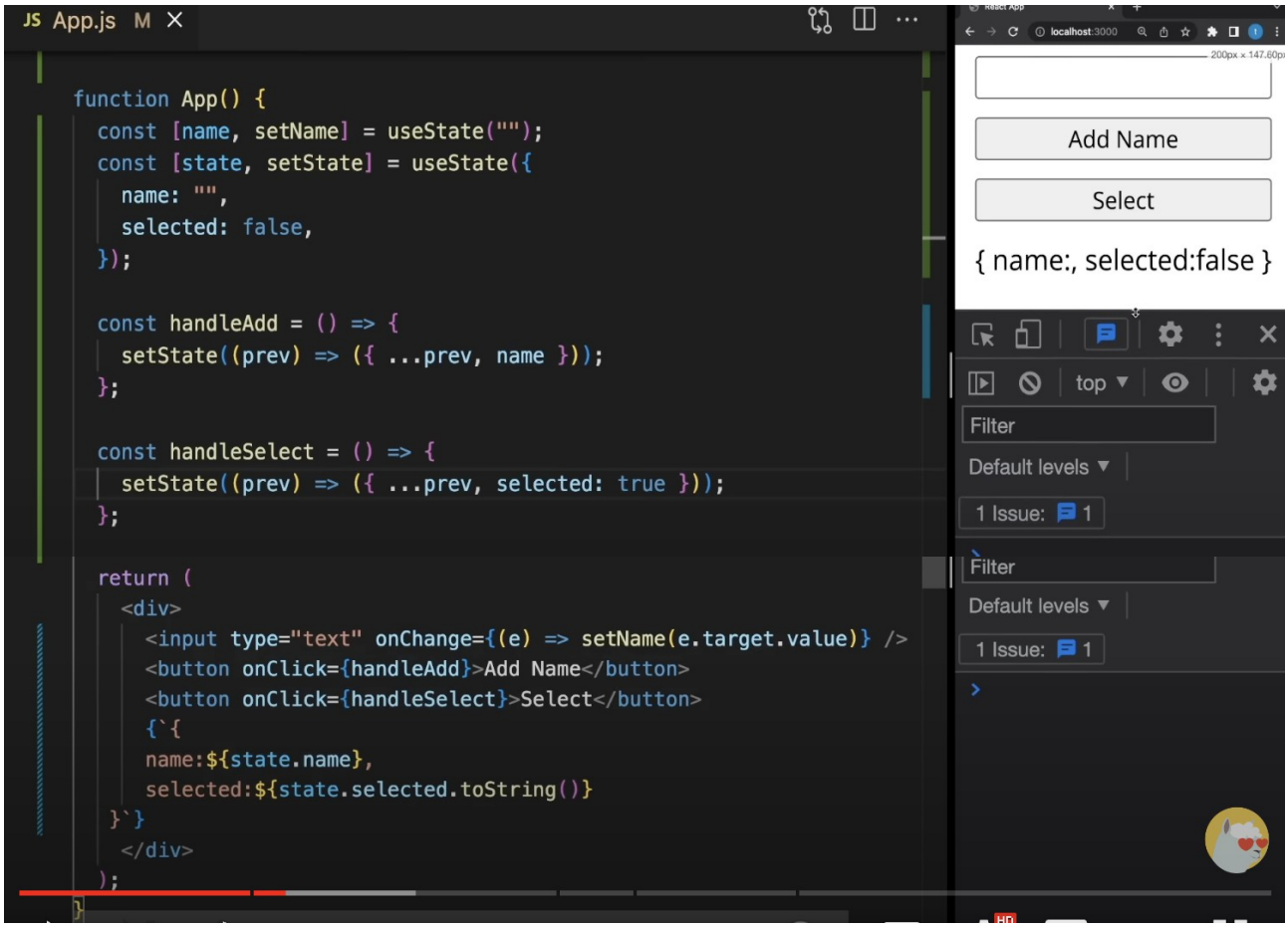


There is an important topic to understand. Working with successfully with useEffect [dependencies] has a lot to do with the type of value

- primitive
- non-primitive

PETER WOLF
< Web Developer & Advisor />

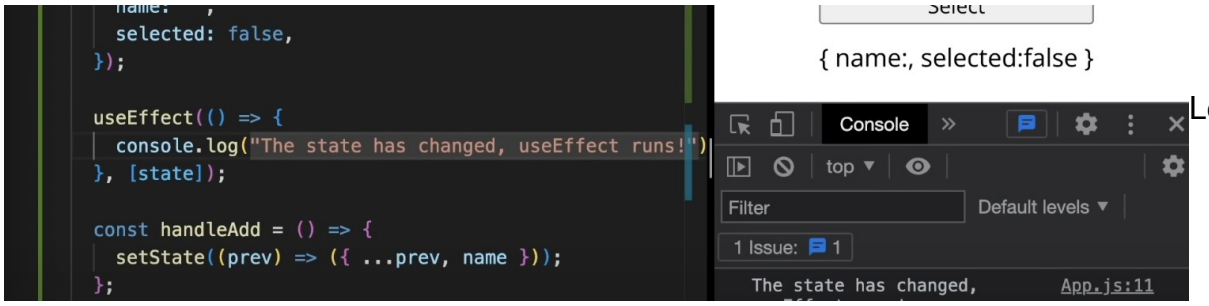
Second Example



We have now an state Object with two properties. {name: „“, selected: false}

in the return we can update the properties...works fine…

lets put an useEffect to the component.



Lets update both values – the useEffect is listening to the state Object and it runs after updating the state.name and state.selected.

BUT if I click on the button to update the state.selected – which is again a false value – the useEffect runs again. But the value didn´t change in the client output.
Same for the name „john“.

WHY IS THAT?

This is related to the primitive and non-primitive values...

Explain primitive and non-primitives.

String, numbers, null, undefined, booleans are primitives

1 === 1 => true
true === true => true
false === false => true
„1“ === „1“ => true

objects and arrays are non-primitive values.

Const x = {name. „john“}
const y = {name: „john“}

x === y => false

non-primitive values refering to different memory spots. The x and y looks the same, but they have different spots in the memory..meaning...they are not equal.

They are 2 different objects

if I refer x = y and I run

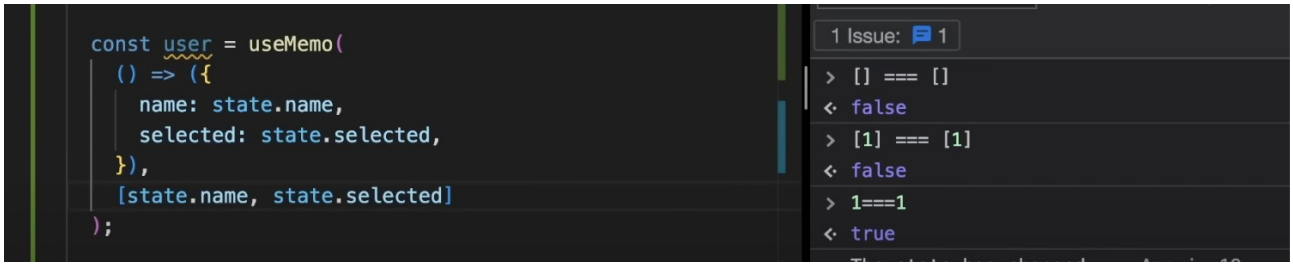
x === y => true

SAME WITH OBJECTS.

In useEffect we can solve this issue in different ways.

PETER WOLF
< Web Developer & Advisor />

UseMemo solution.



Then change the useEffect dependencie

OR

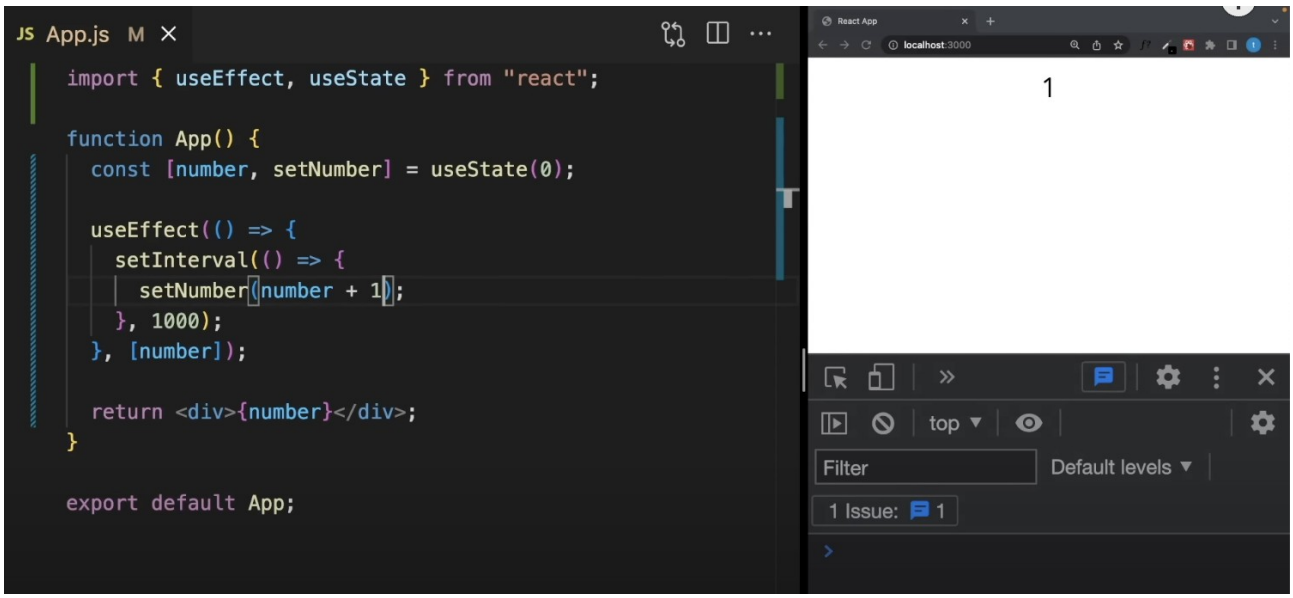
Another way is to wright each dependency single by single…

this is not convenient

both solutions will make the useEffect behavior correct.

PETER WOLF
< Web Developer & Advisor />

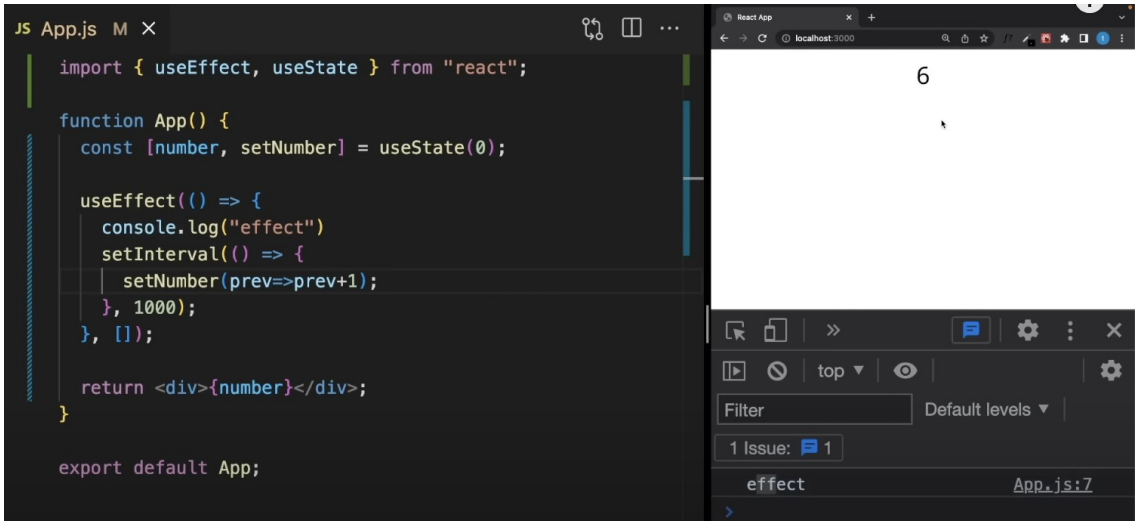
Update the state correct



This component will update every 1000ms the number by 1

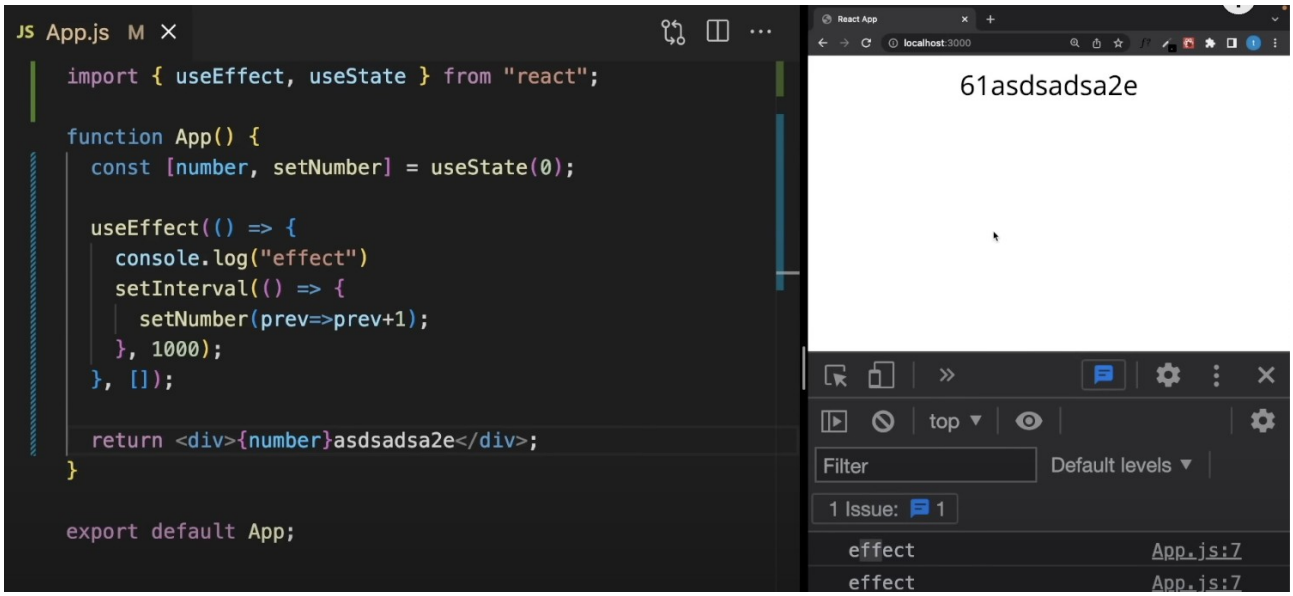
If you have a closer look, you will see a kind of glitch at the number…
you will have now an infinite loop.

To avoid the infinite loop, you have to use an updating function to the number



useEffect CleanUp functions

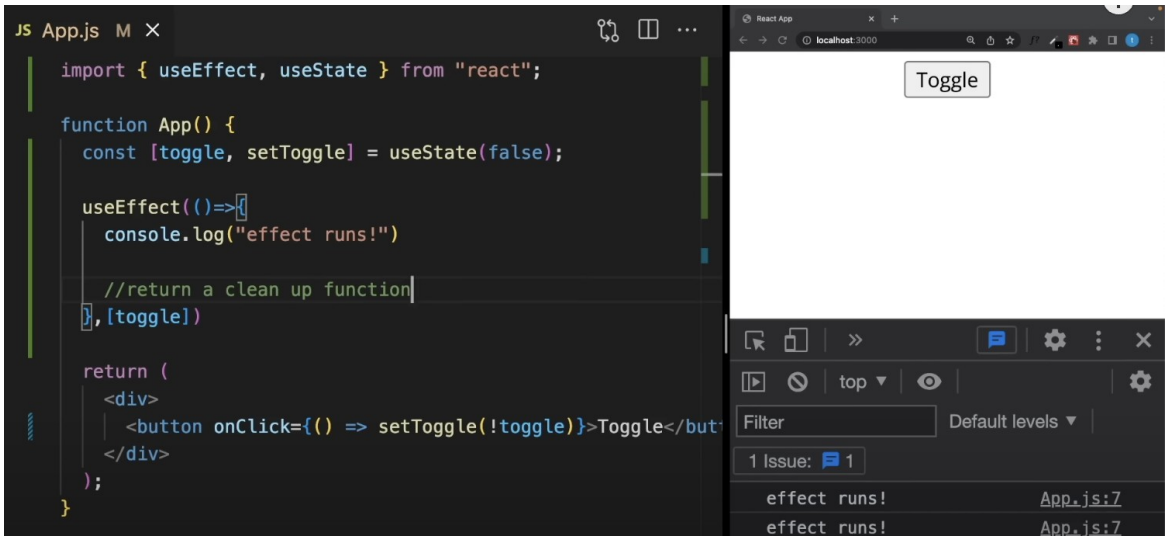
Taking the above intervall example, we added some values to the return manually.



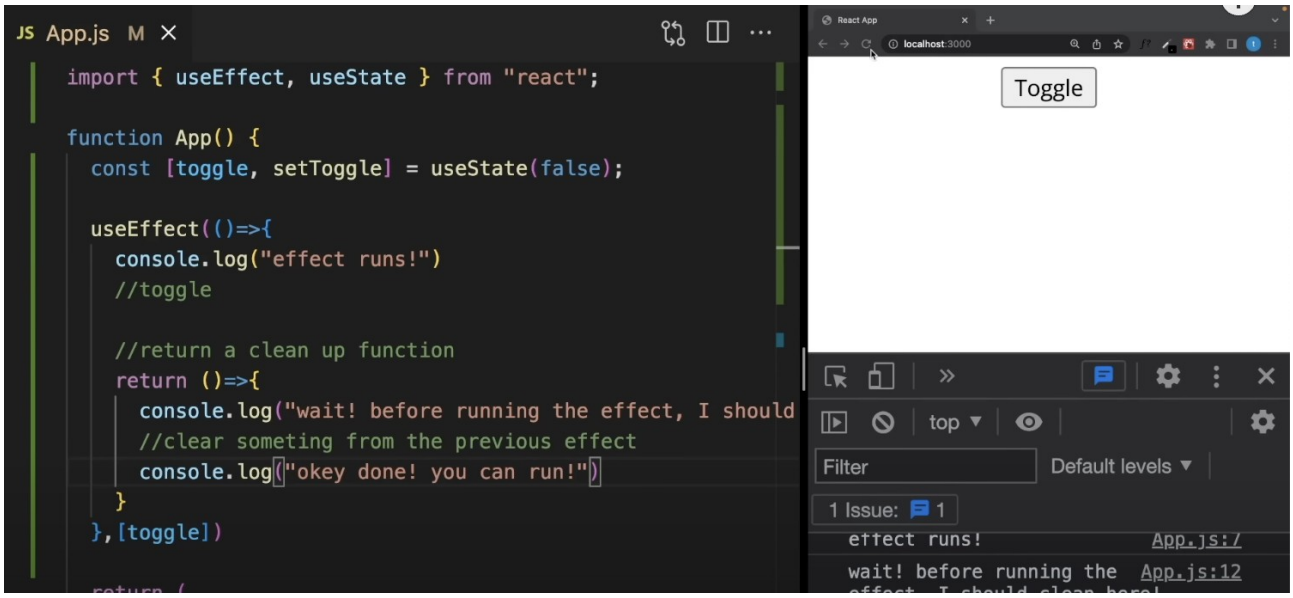
You will see, the output is kind of messed up. The reason for that is, that with each typing of characters we are starting a new intervall.

To clean that up, we will use a cleanup function.

=> new Example for demo CLEANUP :

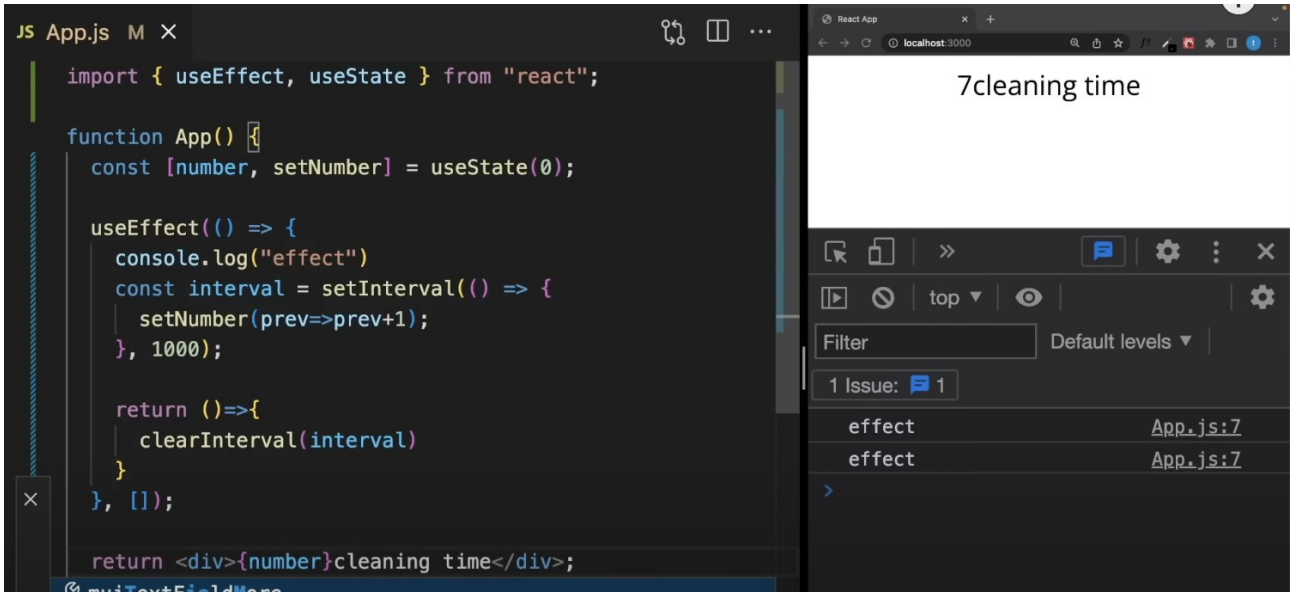


With each toggle click, the useEffect will run.



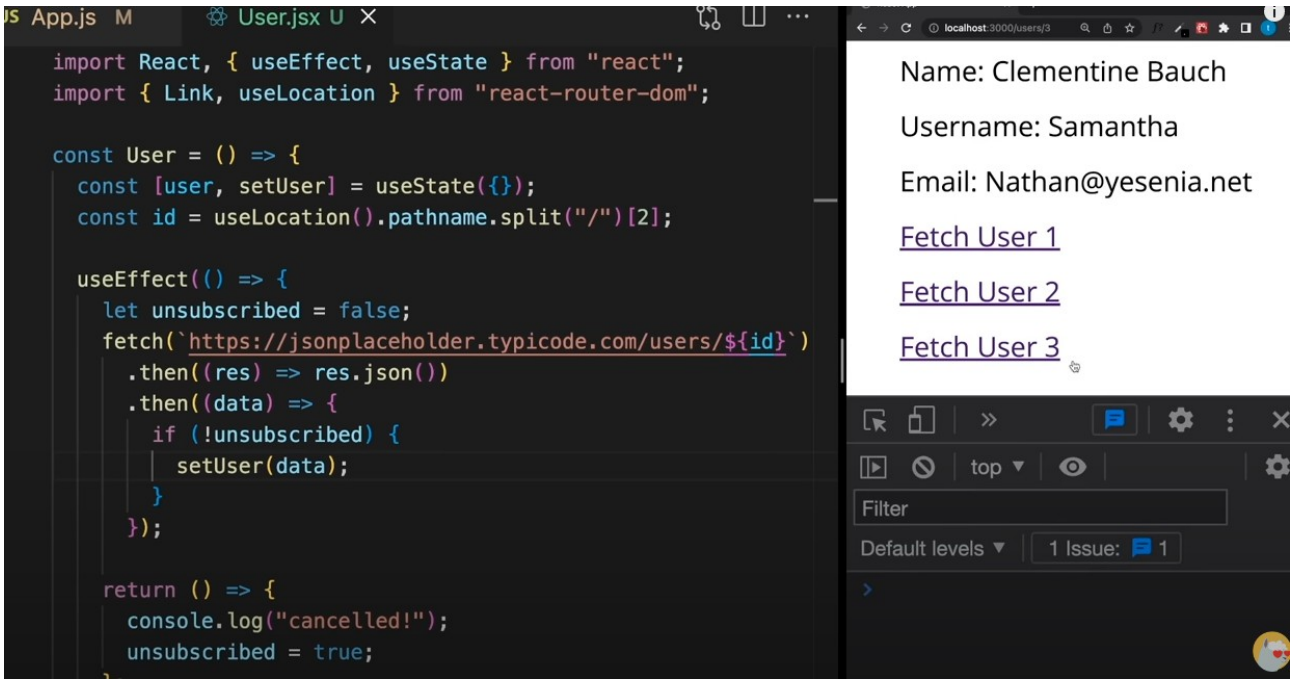
Lets add an cleanup function to demo the behavior

Going back to the interval example...we will add the cleanup function to it



PETER WOLF
< Web Developer & Advisor />

Example user fetching

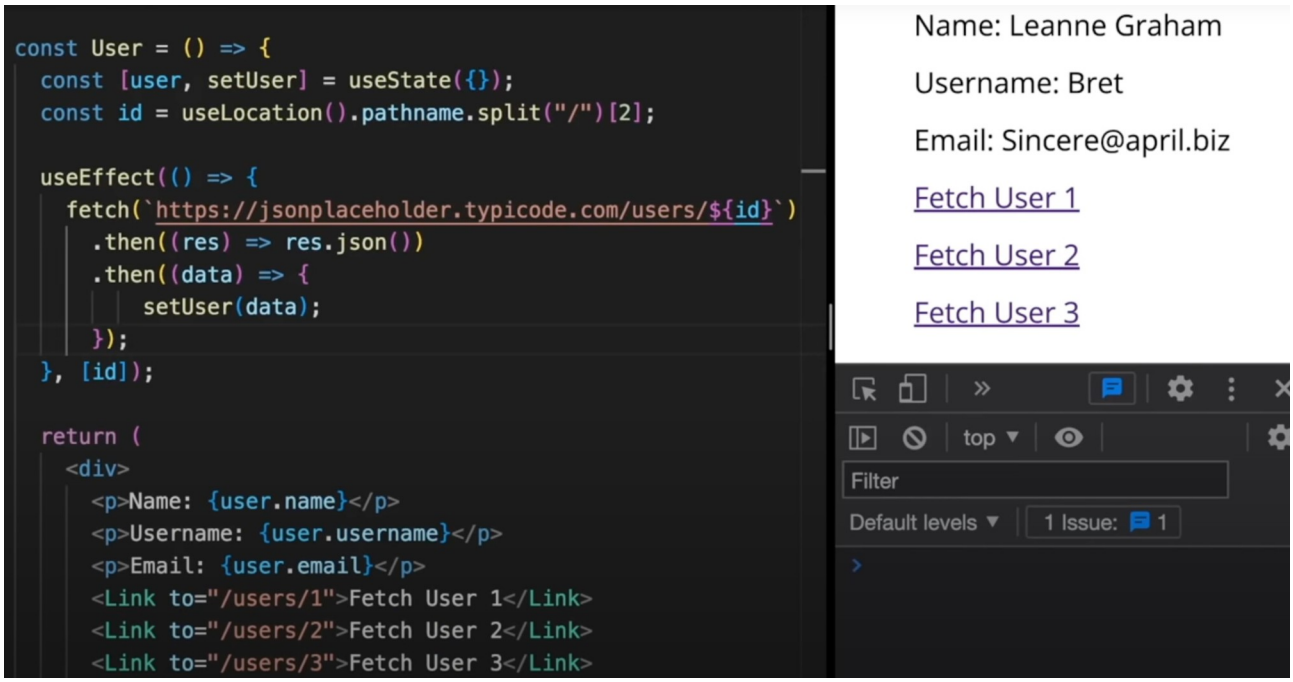


with this example there will be only one user shown.

PETER WOLF
< Web Developer & Advisor />

More professional solution with AbortController()

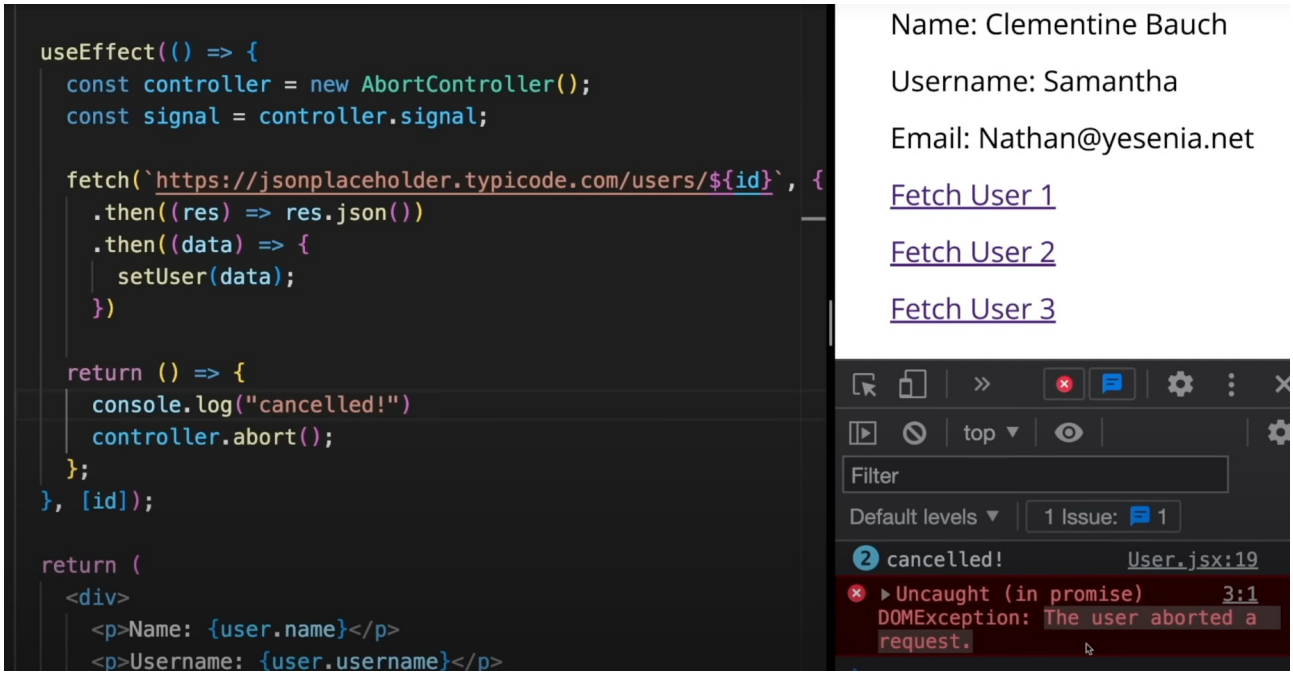
https://youtu.be/QQYeipc_cik?t=1058



Now if you click quick from user one to user two, you will see a big delay.. this is because of the not stopped (- cleaned) request… SOLUTION

PETER WOLF
< Web Developer & Advisor />

Cleanup your request



Now if you click quick, the data is precise